

Git & GitHub Essentials

1. What is Version Control?

Analogy:

Imagine we are building a house. Each time we make a change, we take a snapshot to see how things look, so if anything breaks, we can always revert to a previous state.

Version Control is like keeping track of all those snapshots for our code, so if something goes wrong, we can revert back to a previous version without losing any work.

Real-World Example:

In a **software project**, version control helps manage the codebase's history. For example, imagine 2 peoples are working on a Python web app. They both make changes to the same `index.py` file, and version control ensures that changes can be tracked, merged, or reverted based on what each person does.

2. What is Git?

Analogy:

Think of **Git** as a **time machine** for our code. Every time we make a change, Git records that change. If we make an error, we can “travel” back to the previous version of the code.

Real-World Example:

Let's say we're working on a website and we change the navigation bar style. We use Git to commit that change and push it to GitHub. Later, if a bug is introduced, Git allows us to **revert to the previous working state** by using `git checkout` or `git reset`.

3. What is GitHub?

Analogy:

GitHub is like **cloud storage** for our code. We keep our code on our computer (with Git), but GitHub allows us to store and share it with others online.

Real-World Example:

Some of the people developing a website. They both use GitHub to push their changes, which lets each of their work on the same codebase, independently. GitHub provides features like **Pull Requests**, **Issues**, and **Actions** for collaboration.

4. Why Use Git and GitHub?

Real-World Example:

If we're a developers working on a web app with a team, Git allows us to track the code's history. If someone makes a mistake, we can roll back to the last good version. GitHub is where our code is stored remotely, so everyone on our team can access the latest version.

5. Version Control Systems

Types of Version Control Systems

1. **Local Version Control:** Keeps track of changes on our own computer.
2. **Centralized Version Control:** Has a single, central server where all changes are committed.
3. **Distributed Version Control:** Git allows each developer to have a full copy of the repository, enabling more flexibility and redundancy.

6. Git Basics

Installation

- **Windows:** Download and install Git from git-scm.com.
- **Ubuntu:** Run the following:

```
sudo apt update  
sudo apt install git
```

Configuration

Set **username** and **email**:

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example"
```



Initializing a Repository

To start tracking a project:

```
git init
```

7. GitHub: Creating and Cloning Repositories

Creating a Repository on GitHub:

1. Go to GitHub and create a new repository.
2. Log in to GitHub: Open GitHub and sign in with account credentials.
3. Create a New Repository:
 - Click the **New** button or go to the “Repositories” tab and select “New.”
 - Provide a name for your repository (e.g., `my-project`).
 - Choose to make it **Public** or **Private**, and decide whether to initialize it with a README, `.gitignore` , or license (optional).
 - Click **Create Repository**.

Copy the URL provided to clone it locally.

Copy and Clone URL:

- After the repository is created, we’ll see the repository page.
- Click the **Code** button and choose between the **HTTPS** or **SSH URL** to clone the repository.

1. Clone the Repository Locally:

Use the copied URL to clone the repository

```
git clone <URL>
```

Note:

- **HTTPS** is simpler for beginners and requires our GitHub username and password for every operation (or a personal access token).
- **SSH** is preferred for advanced users who have configured SSH keys, as it eliminates the need to enter credentials repeatedly.

Cloning a Repository from GitHub

Clone a repository to our local machine:

```
git clone https://github.com/username/repository.g
```

8. Branching and Merging

Types of Branches

1. Main Branch:

- This is the default branch, often named **main** or **master**.
- It contains the stable, production-ready code.
- Direct commits to the main branch are rare; changes are merged from other branches after review.

2. Feature Branches:

- Used to develop new features or enhancements.
- Typically created from the **main** branch.
- Example: `feature/login-page` , `feature/user-dashboard` .
- These branches are merged into the main branch after the feature is complete and reviewed.

3. Hotfix Branches:

- Created for urgent fixes to the production code.
- Usually branched off from **main** and merged back into **main** (and often **develop**, if present).
- Example: `hotfix/security-patch` .

4. Bugfix Branches:

- Used to fix identified bugs in the code.
- Similar to feature branches but specifically for resolving issues.
- Example: `bugfix/form-validation` , `bugfix/api-endpoint` .

5. Release Branches:

- Used to prepare a new release.
- Created from the **main** or **develop** branch, depending on the workflow.
- Example: `release/v1.0` .
- Allows final testing, documentation, and version updates before merging into **main**.

6. Develop Branch:

- Acts as an integration branch for feature branches.
- Used in workflows like **Git Flow** to maintain a stable base for testing and integration.
- Example: `develop` .

7. Experimental Branches:

- Used for trying out new ideas or concepts without affecting the main workflow.
- Example: `experiment/ai-integration` .

Note: The exact types of branches you use may depend on team's workflow

Merging Branches

To merge our feature branch back to the main branch:

```
git checkout main  
git merge feature-branch
```

```
git status  
git add <file>  
git commit -m "Resolved merge conflict"
```

Merge Conflicts

A merge conflict happens when Git can't automatically combine changes from two branches because they modify the same part of a file or one branch deletes something the other modifies.

When this occurs:

- Git marks the conflicted files and pauses the merge.
- We will review the differences, choose which changes to keep (or combine them), and save the file.
- Once resolved, we can complete the merge by committing the changes.

Merge conflicts are normal when working multiple developers and ensure that everyone's updates are properly integrated.

9. Advanced Git Topics

GitOps

GitOps is the practice of using **Git** as the source of truth for infrastructure. Any change made to the Git repository is automatically applied to our infrastructure.

Rebasing

Rebase allows us to move our branch's base to a different commit:

```
git checkout feature-branch  
git rebase main
```

Git Stash

Temporarily save changes that are not ready to be committed:

```
git stash
```

Git Bisect

Git Bisect helps find the commit that introduced a bug. Git will guide us through a binary search process:

```
git bisect start  
git bisect bad  
git bisect good <commit_hash>
```

10. Git and GitHub Workflows

Forking and Pull Requests

What is Forking?

Forking is the process of creating our own copy of someone else's repository on GitHub. This allows us to experiment, make changes, or contribute without affecting the original project.

Fork a repository to create our own copy.

What is a Pull Request?

A pull request is a way to propose changes from our fork to the original repository. It allows repository maintainers to review, discuss, and merge our contributions.

Create a **pull request** to propose changes from our fork to the original repository.

Collaborating with a Team on GitHub

1. Create a Shared Repository:

- One team member creates a repository and shares the **repository link** with the team.
- The repository can be public or private, depending on project's needs.

2. Add Collaborators:

- The repository owner can go to **Settings > Collaborators and Teams** and invite team members to collaborate.
- Each team member must accept the invitation to gain access.

1. Clone the Repository:

- Team members can clone the repository using the provided HTTPS or SSH link

Collaborating on Projects

1. Cloning a repository.

```
git clone <repository-url>
```

1. Creating a new branch for each feature.

```
git checkout -b feature-branch-name
```

Work on Individual Features: Each member should create a **branch** for their task or feature

```
git checkout -b feature-branch-name
```

Committing changes locally.

```
git add .  
git commit -m "Describe changes"
```

Pushing changes to the remote repository.

```
git push origin feature-branch-name
```

Creating a pull request for review.

- Go to the repository on GitHub and click **Pull Requests > New Pull Request**.
- Select branch and compare it with the main branch (or another target branch).

- Add a title and description, then click **Create Pull Request**.

Code Review and Merge:

- Team members can review the PR, suggest changes, or approve it.
- Once approved, the repository owner (or a designated team member) can **merge** the changes into the main branch.

11. Git Hosting Options

GitHub

GitHub provides cloud-based hosting and collaboration for Git repositories. It is the most popular platform for open-source and private projects.

Other Hosting Options

1. GitLab:

- A Git repository manager that offers both self-hosted and cloud-hosted options.
- Known for built-in CI/CD tools, issue tracking, and DevOps features.

2. Amazon S3 (AWS):

- A scalable object storage service by Amazon Web Services.

- Often used for hosting static websites, storing backup files, or managing assets for web applications.
- Supports high durability, availability, and integration with other AWS services for enhanced workflows.

12. Git and GitHub Commands Cheat Sheet

****Repository Setup****

<code>git init</code>	: Initialize a Git repository
<code>git clone <url></code>	: Clone a repository from a remote

****Basic Operations****

<code>git add <file></code>	: Stage changes for commit
<code>git add .</code>	: Stage all changes for commit
<code>git commit -m "message"</code>	: Commit staged changes with a message
<code>git push</code>	: Push local changes to a remote
<code>git pull</code>	: Fetch and merge changes from a remote

****Branching Commands****

<code>git branch</code>	: List all branches
<code>git branch <name></code>	: Create a new branch.
<code>git checkout <branch></code>	: Switch to another branch
<code>git merge <branch></code>	: Merge another branch into the current
<code>git rebase <branch></code>	: Rebasing the current branch onto another
<code>git branch -d <branch></code>	: Delete a branch.


```
**History and Stashing**  
git log                : View the commit history.  
git stash              : Temporarily save changes  
git stash apply        : Apply stashed changes.  
  
**Remote Repositories**  
git remote -v          : View remote repositories  
git remote add <name> <url> : Add a new remote repository  
git fetch              : Fetch updates from a remote repository  
git diff               : it is used to show the differences between files
```

13. What is Git GUI?

Imagine we have a **super-organized filing cabinet** (our Git repository).

- **Using the command line** to work with Git is like **opening the cabinet manually**, pulling out folders, rearranging papers, and labeling everything ourself. It's powerful and precise but requires us to know exactly what we're doing.
- **Using a Git GUI**, on the other hand, is like having a **digital filing assistant** with a touchscreen interface. We can:
 - Tap to **add papers to a folder** (staging files).
 - Swipe to see **what changes we've made** (file differences).

- Press a button to **share our files** with others (pushing to a remote repository).

It's more intuitive and visual. We don't need to remember commands because everything is laid out for us with buttons and menus.

Why Use Git GUI?

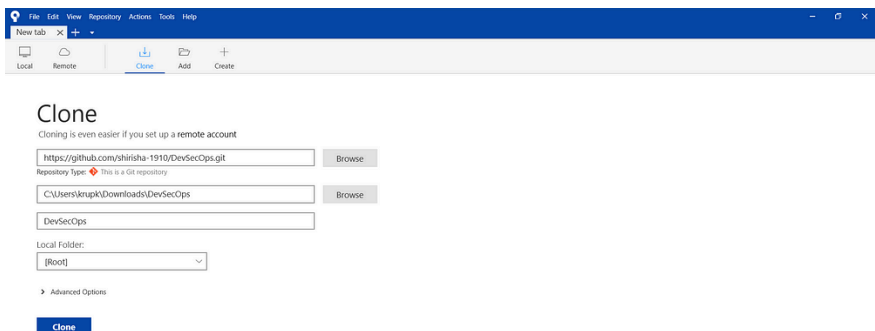
It's like upgrading from **manual filing** to a **fancy filing app**:

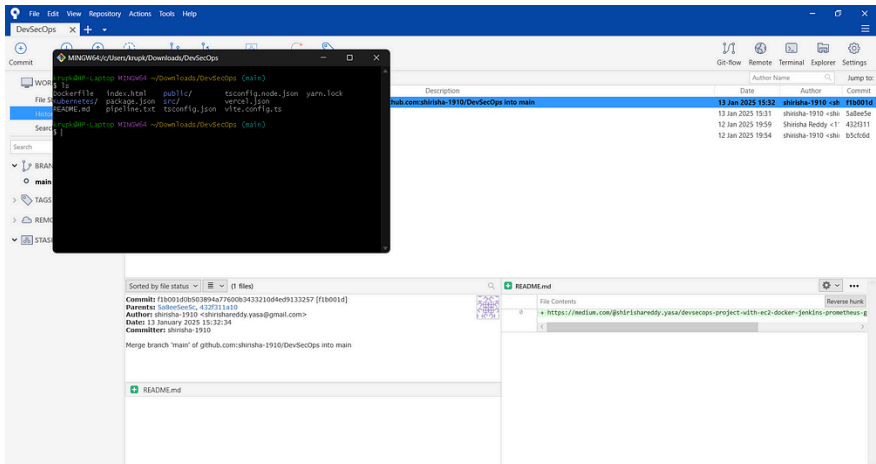
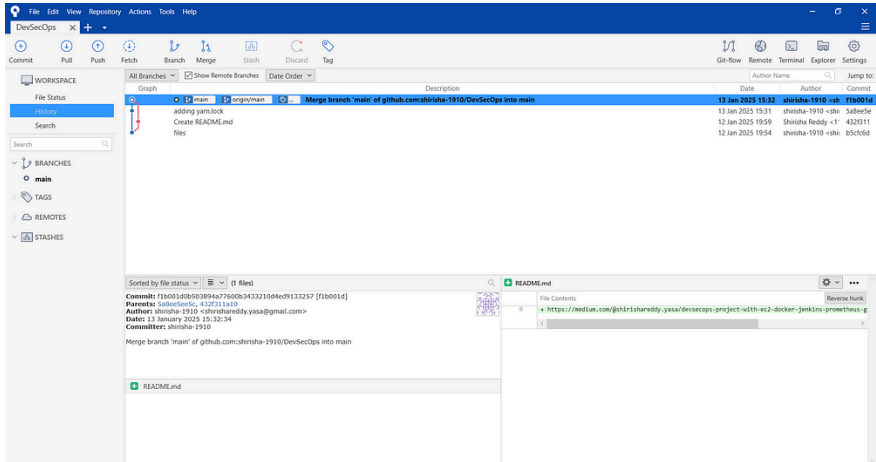
- **Visualizing Changes:** Instead of hunting through papers to spot differences, the GUI highlights them for us.
- **Branching:** Think of branches as separate stacks of papers; with a GUI, we can see and switch between them easily.
- **Conflict Resolution:** If two people accidentally write on the same document, the GUI helps us to resolve the mix-up with a guided interface.

Git GUI makes using Git easier, just like using a touchscreen makes interacting with technology simpler. While the **manual approach (command line)** gives us ultimate control, the **assistant (GUI)** is perfect for simplifying and speeding up tasks.

In a Git GUI, **code versioning** refers to managing and tracking changes to our code visually, just like in the command line. Here's how it works in a Git GUI:

1. **Commit History:** View a list of all commits with their messages, timestamps, and changes made.
2. **File Changes:** See specific differences (added/removed lines) in files between versions.
3. **Branching:** Create, switch, and merge branches visually to maintain different versions of our code.
4. **Revert Changes:** Roll back to a previous commit or undo specific changes without using CLI commands.
5. **Conflict Resolution:** Handle merge conflicts with a user-friendly interface highlighting differences.





Difference Between Git, GitHub, and Git GUI

- **Git:** A tool to manage and track changes in our code locally.

- **GitHub:** A cloud platform to host and collaborate on Git repositories.
- **Git GUI:** A visual tool to interact with Git without using the command line.